HACK THE BOX MEETUP • WORKSHOP GUIDE

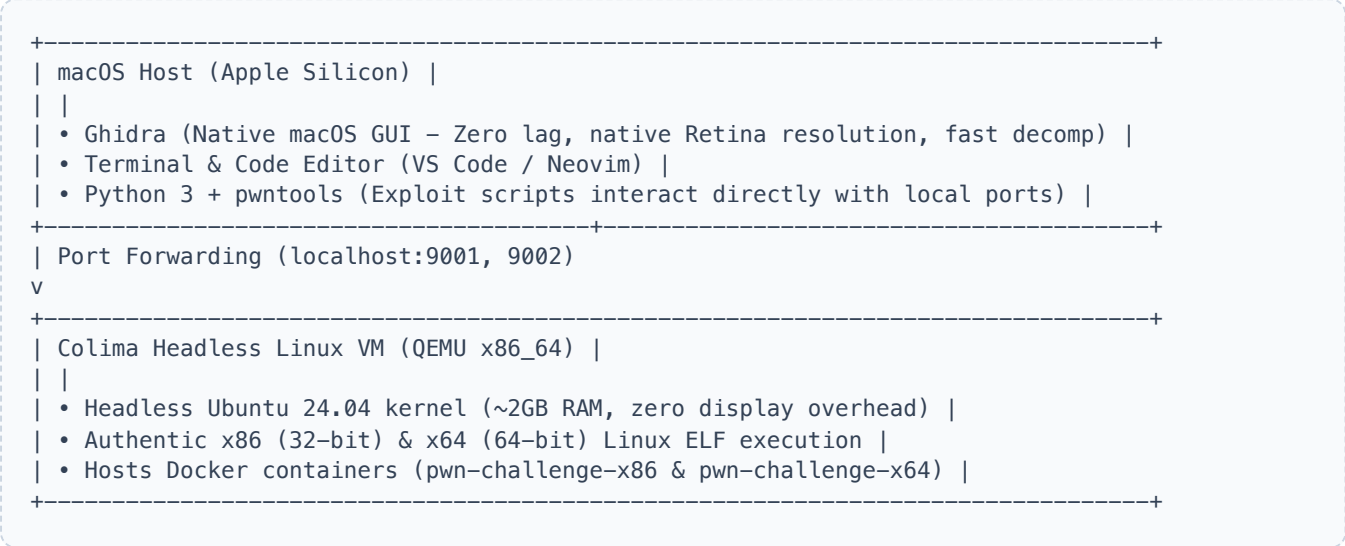
Apple Silicon Setup Guide (M1/M2/M3/M4)

Lightweight Binary Exploitation (PWN) Workshop Setup • Host: macOS • Execution: Colima Linux (x86_64)

1. Overview & Architecture

Running full desktop Linux virtual machines (e.g. Kali Linux GUI inside VMware Fusion or UTM) under x86_64 emulation on Apple Silicon is extremely demanding: it consumes **8GB+ RAM**, **40–50GB of disk space**, and suffers from heavy cursor/GUI lag.

To solve this, we use a **Lightweight Split Architecture**:



Host OS: All reverse engineering and scripting happens natively on macOS.
Colima: Linux is used *strictly for execution and containers*, keeping your system fast and responsive.

2. Prerequisites

Ensure you have Homebrew installed on macOS:

```
/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
```

3. Step 1: Install Ghidra on macOS Host

Installing Ghidra directly on macOS provides native hardware acceleration and seamless window management:

```
# 1. Install Ghidra and its OpenJDK 21 dependency
brew install ghidra

# 2. Create a symlink so typing 'ghidra' launches it from any terminal
ln -sf /opt/homebrew/bin/ghidraRun /opt/homebrew/bin/ghidra
```

> **✔ Verification**
>
> Run `which ghidra` in your terminal. It should return `/opt/homebrew/bin/ghidra`. You can now launch Ghidra anytime simply by typing `ghidra`.

4. Step 2: Install Colima & QEMU x86_64

Install Colima and the required virtualization components via Homebrew:

```
brew install colima qemu lima lima-additional-guestagents docker docker-compose
```

> **⚠ Important: Apple Silicon Lima Guest Agent Link**
>
> On Apple Silicon, Lima only includes the ARM64 guest agent by default. The `lima-additional-guestagents` formula provides the x86_64 guest agent. Link it into Lima's Cellar directory:
>
> ```
> ln -sf /opt/homebrew/share/lima/lima-guestagent.*.gz /opt/homebrew/Cellar/lima/*/share/lima/
> ```

5. Step 3: Launch Colima in x86_64 Mode

Start the lightweight x86_64 Linux environment:

```
colima start --arch x86_64 --vm-type qemu
```

Verify that your Docker CLI now communicates with an x86_64 Linux daemon:

```
$ colima status
INFO[0000] colima is running using QEMU
INFO[0000] arch: x86_64
INFO[0000] runtime: docker

$ docker info | grep Architecture
 Architecture: x86_64
```

6. Step 4: Clone the Workshop Repository

```
git clone https://github.com/danishhakim/htb-meetup-pwn.git
cd htb-meetup-pwn
```

7. Step 5: Configure Python & Pwntools on macOS

Set up an isolated Python virtual environment on macOS to run exploit scripts:

```
# 1. Create and activate virtual environment
python3 -m venv ~/.pwn-env
source ~/.pwn-env/bin/activate

# 2. Install pwntools
pip install --upgrade pwntools
```

8. Step 6: Running & Exploiting Mini-CTF Challenges

The CTF challenges run inside Docker containers hosted by Colima, with network ports forwarded to `localhost`.

1. Start the Containers:

```
cd mini-ctf
docker compose up -d
```

2. Extract Challenge Binaries:

Extract the compiled binaries to analyze them on your Mac:

```
chmod +x extract-binaries.sh
./extract-binaries.sh
```

This creates `binaries/challenge_x86` and `binaries/challenge_x64`.

3. Reverse Engineer in Ghidra (macOS):

```
ghidra &
```

1. Open Ghidra and create a new project.
2. Import `binaries/challenge_x86` and `binaries/challenge_x64`.
3. Identify buffer sizes, vulnerabilities, and target functions (e.g. `get_flag()`).

4. Execute Exploits from macOS:

Run the solve scripts natively from your macOS terminal:

```
# Test the 64-bit challenge:
python3 x64/solve.py
# Output: 🚩 HTB{x64_st4ck_sm4sh1ng_pr0}

# Test the 32-bit challenge:
python3 x86/solve.py
# Output: 🚩 HTB{x86_buff3r_0v3rfl0w_m4st3r}
```

5. Tear Down:

When finished with the CTF:

```
docker compose down
```

9. Step 7: Guided Hands-on Challenges

The guided practice challenges are located in `handson/`:

```
# 1. Rebuild binaries (if needed) via Docker
docker run --rm -v "$PWD:/workspace" -w /workspace test-x86 ./handson/build.sh

# 2. Test hands-on solve scripts
cd handson/x64
python3 solve.py    # Output: FLAG{x64_r3t2w1n_b4s1cs}

cd ../x86
python3 solve.py    # Output: FLAG{x86_r3t2w1n_b4s1cs}
```

10. Colima Lifecycle Management

Command	Description
`colima stop`	Pauses the x86_64 VM to conserve battery and CPU when not in use.
`colima start`	Resumes the instance with all saved state.
`colima ssh`	Opens an interactive bash terminal inside the x86_64 Ubuntu VM.
`colima delete -f`	Destroys the instance and frees all disk space.

11. Troubleshooting Reference

Error / Symptom	Root Cause	Resolution
`guest agent binary could not be found for Linux-x86_64`	Lima lacks x86_64 guest agent on Apple Silicon.	`brew install lima-additional-guestagents` and link into Cellar directory.
`ghidra: command not found`	Homebrew names the binary `ghidraRun`.	`ln -sf /opt/homebrew/bin/ghidraRun /opt/homebrew/bin/ghidra`
`Cannot connect to Docker daemon`	Colima stopped or wrong context.	Run `colima start` and `docker context use colima`.
`ModuleNotFoundError: No module named 'pwn'`	Python virtualenv not activated.	Run `source ~/.pwn-env/bin/activate`.